

BITÁCORA DE IA

La integración de herramientas de asistencia basada en inteligencia artificial generativa constituyó un eje metodológico fundamental para el desarrollo del simulador, permitiendo realizar el diagnóstico de la estructura de archivos, corregir errores de compatibilidad y automatizar el entorno de pruebas.

Prompts utilizados

El proceso de desarrollo dio inicio formal una vez que se consolidó el panorama completo del repositorio. El primer bloque de interacción con la inteligencia artificial se activó con la finalidad de establecer la arquitectura base, documentando textualmente el siguiente requerimiento técnico:

- "Perfecto, ahora tengo el panorama completo. Voy a generar todo el proyecto de una vez — los archivos que faltan y los que necesitan implementación real. El plan es: Makefile corregido (el doble asterisco no funciona en GNU Make). Headers faltantes (queue, circular_queue, linked_list, doubly_linked_list, scheduler, algorithms). Implementaciones C completas (stack, queue, circular_queue, linked_list, doubly_linked_list, schedulers, memory, algorithms). El archivo main.c del proyecto (no el de tests). Scripts Python completos."

Respuestas generadas

En respuesta al requerimiento anterior, el entorno de desarrollo generó de forma automatizada las estructuras modulares correspondientes a diez archivos de configuración y treinta y un componentes en lenguaje C, empaquetando el código final en un archivo comprimido de extensión ZIP. La herramienta detalló el inventario de elementos generados bajo los siguientes términos:

- "Reemplazar estos archivos que ya tienes (están corregidos y completados): Makefile — fix del glob de doble asterisco, ahora lista archivos explícitamente. El archivo src/stack.c — implementación completa con las operaciones push, pop, peek y destroy. El archivo src/memory/first_fit.c — implementación real de First Fit, Best Fit, Worst Fit, Free, Coalesce y Compact. El archivo src/memory/coalescence.c — el cual ya no está vacío. El archivo src/scheduler/round_robin.c — implementación completa con quantum y cola circular. El archivo scripts/benchmark.py — ahora genera el archivo CSV y dos gráficas. El archivo scripts/generate_process.py — ahora exporta el formato CSV que el lenguaje C puede leer correctamente."
- "Agregar estos archivos nuevos: En la carpeta include se incorporan queue.h, circular_queue.h, linked_list.h, doubly_linked_list.h, scheduler.h, algorithms.h,

logger.h, timer.h y parser.h. En la carpeta src se agregan queue.c, circular_queue.c, linked_list.c, doubly_linked_list.c y main.c. En la carpeta src/scheduler se añaden fifo.c y sjf.c. En la carpeta src/memory se integran best_fit.c, worst_fit.c y compactation.c. En la carpeta src/algorithms se incorporan todos los métodos correspondientes a brute_force, greedy, backtracking, divide_conquer, dp_bottomup, dp_topdown e incremental. En la carpeta src/utils se añaden logger.c, timer.c y parser.c. Finalmente, en la carpeta scripts se agregan graphs.py y validate_results.py."

Errores encontrados

Antes de realizar la intervención de programación profunda, la herramienta proporcionó un estado de situación crítico para identificar de forma explícita las carencias, errores de ubicación y omisiones del proyecto original:

- "Perfecto, ya veo el estado real del repositorio. Déjame hacer un diagnóstico claro respecto al estado actual del repositorio. Lo que existe con contenido parcial: En include únicamente memory_manager.h, process.h y stack.h. En src/memory se encuentran coalescence.c y first_fit.c. En src/scheduler está round_robin.c. El archivo src/stack.h está en el lugar equivocado, debería estar en la carpeta include. En scripts se localizan benchmark.py y generate_process.py. En tests se ubican los cinco archivos de prueba junto con main.c."
- "Lo que falta completamente: En include hacen falta queue.h, circular_queue.h, linked_list.h, doubly_linked_list.h, scheduler.h y algorithms.h. En src faltan queue.c, circular_queue.c, linked_list.c, doubly_linked_list.c y main.c. En src/scheduler faltan fifo.c y sjf.c. En src/memory faltan best_fit.c, worst_fit.c y compactation.c. En las carpetas src/algorithms y src/utils faltan todos los archivos correspondientes."

Correcciones realizadas

Para solventar las omisiones detectadas en el diagnóstico de errores, se autorizó la reestructuración completa del Makefile debido a que las reglas originales de compilación utilizaban un formato de búsqueda global inviable para el entorno GNU. Asimismo, se reubicaron las cabeceras mal posicionadas y se programaron desde cero los archivos fuentes faltantes de las colas, las listas enlazadas, los planificadores FIFO y SJF, y los algoritmos avanzados de optimización para unificar las interfaces del sistema operativo.

Validaciones realizadas.

La bitácora cierra con el requerimiento de control planteado por la inteligencia artificial para asegurar la lectura correcta de las interfaces preexistentes sin romper el código funcional del equipo de desarrollo:

- "Para poder ayudarte bien necesito ver el contenido de los archivos que ya existen. ¿Puedes subir o pegar el contenido de estos archivos clave? Los archivos son: include/process.h, include/memory_manager.h, include/stack.h, el archivo src/stack.h que está mal ubicado, src/scheduler/round_robin.c, src/memory/first_fit.c y el Makefile. Con eso arrancamos sin romper lo que ya tienes construido."